Week 7: Transactions, MVCC, and PostgreSQL Internals

Phase 3: Internals & Operations | Week 7 of 12

Week Overview

This week demystifies what happens inside PostgreSQL during concurrent operations. Understanding MVCC, isolation levels, and locking is the difference between writing correct concurrent applications and introducing subtle data corruption bugs. These topics appear in every senior-level interview.

Focus: You cannot reason about concurrency bugs without understanding MVCC. This week changes how you think about databases permanently.

Learning Objectives

By the end of this week, you will be able to:

- Explain MVCC and why PostgreSQL uses it.
- Describe the four transaction isolation levels and their anomaly guarantees.
- Identify and prevent deadlocks.
- Distinguish row-level locks, table-level locks, and advisory locks.
- Understand what happens to dead rows and why VACUUM is needed.
- Explain transaction ID wraparound and why it's catastrophic.
- Use SKIP LOCKED and NOWAIT for queue implementations.
- Write concurrent-safe upsert patterns with `INSERT ... ON CONFLICT`.

Required Reading

- `09_Transactions_Concurrency/` — All files
- `10_PostgreSQL_Internals/` — All files

Daily Schedule

Monday — Transactions and Isolation Levels (60 min)

Topics:

- BEGIN, COMMIT, ROLLBACK, SAVEPOINT
- Isolation anomalies: dirty read, non-repeatable read, phantom read
- PostgreSQL isolation levels: READ COMMITTED (default), REPEATABLE READ, SERIALIZABLE
- When to use each level
- Implicit vs. explicit transactions

```
-- Transaction basics
BEGIN;
  INSERT INTO accounts (id, balance) VALUES (1, 1000);
  INSERT INTO accounts (id, balance) VALUES (2, 500);
```

```
  -- If anything fails here, ROLLBACK returns to state before BEGIN
COMMIT;

-- SAVEPOINT for partial rollback
BEGIN;
  UPDATE accounts SET balance = balance - 100 WHERE id = 1;
  SAVEPOINT before_credit;
  UPDATE accounts SET balance = balance + 100 WHERE id = 2;
  -- Oops, wrong account
  ROLLBACK TO SAVEPOINT before_credit;
  UPDATE accounts SET balance = balance + 100 WHERE id = 3;
COMMIT;

-- READ COMMITTED: default, vulnerable to non-repeatable reads
-- Session 1:
BEGIN;
SELECT balance FROM accounts WHERE id = 1;  -- Returns 1000

-- Session 2 (different terminal) commits while Session 1 is still open:
UPDATE accounts SET balance = 500 WHERE id = 1;
COMMIT;

-- Back in Session 1:
SELECT balance FROM accounts WHERE id = 1;  -- Returns 500! (non-repeatable read)
COMMIT;

-- SERIALIZABLE: strongest guarantee
BEGIN ISOLATION LEVEL SERIALIZABLE;
-- All reads in this transaction see a snapshot consistent as of BEGIN
-- Any serialization conflict causes ROLLBACK with serialization_failure error
```

Tuesday — MVCC Deep Dive (90 min)

Topics:

- How MVCC stores multiple row versions (xmin, xmax)
- Snapshots: which transactions are visible
- Dead tuples: what happens after UPDATE or DELETE
- VACUUM: removing dead tuples, updating visibility map
- autovacuum: the background cleanup process
- HOT (Heap Only Tuple) updates for efficiency
- Transaction ID (XID) and wraparound

```
-- Inspect row versions (requires pageinspect extension)
CREATE EXTENSION IF NOT EXISTS pageinspect;

CREATE TABLE mvcc_demo (id INTEGER, name TEXT);
INSERT INTO mvcc_demo VALUES (1, 'Alice');

-- Check the raw tuple xmin (transaction that created it)
SELECT lp, t_xmin, t_xmax, t_data
```

```
FROM heap_page_items(get_raw_page('mvcc_demo', 0));

UPDATE mvcc_demo SET name = 'Alicia' WHERE id = 1;

-- Now there are TWO versions of row 1
SELECT lp, t_xmin, t_xmax, t_data
FROM heap_page_items(get_raw_page('mvcc_demo', 0));
-- Old version has t_xmax = your UPDATE transaction ID
-- New version has t_xmin = your UPDATE transaction ID

-- Dead tuple accumulation
SELECT relname, n_live_tup, n_dead_tup, last_vacuum, last_autovacuum
FROM pg_stat_user_tables WHERE relname = 'orders';

-- Force vacuum and check bloat
VACUUM VERBOSE orders;

-- Transaction ID watermark
SELECT age(datfrozenxid), datname FROM pg_database;
-- If age > 2 billion, database risks transaction ID wraparound!
```

Wednesday — Locking Mechanics (90 min)

Topics:

- Row-level locks: FOR UPDATE, FOR SHARE, FOR NO KEY UPDATE, FOR KEY SHARE
- Table-level lock modes: ACCESS SHARE → ACCESS EXCLUSIVE
- Lock queue and wait ordering
- Deadlock detection and resolution
- Advisory locks: `pg_advisory_lock`, `pg_advisory_xact_lock`
- SKIP LOCKED for job queues
- NOWAIT to fail fast instead of waiting

```
-- Row-level locks
BEGIN;
SELECT * FROM orders WHERE id = 1 FOR UPDATE;
-- No other transaction can update/delete row id=1 until this COMMIT

-- FOR SHARE (allows other readers but blocks writers)
SELECT * FROM accounts WHERE id = 1 FOR SHARE;

-- SKIP LOCKED: perfect for job queues
CREATE TABLE job_queue (
    job_id     BIGSERIAL PRIMARY KEY,
    payload    JSONB,
    status     VARCHAR(20) DEFAULT 'pending',
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Worker process: pick one job without blocking others
BEGIN;
```

```sql
UPDATE job_queue
SET status = 'processing'
WHERE job_id = (
    SELECT job_id FROM job_queue
    WHERE status = 'pending'
    ORDER BY created_at
    LIMIT 1
    FOR UPDATE SKIP LOCKED   -- Critical: skip jobs other workers have
)
RETURNING *;

-- Advisory locks for application-level coordination
-- Useful for: preventing concurrent report generation, distributed cron jobs
SELECT pg_advisory_lock(12345);    -- Session-level (must call pg_advisory_unlock)
SELECT pg_advisory_xact_lock(12345); -- Transaction-level (auto-releases at COMMIT)

-- Deadlock demonstration (run in two sessions):
-- Session 1: BEGIN; UPDATE accounts SET balance = 0 WHERE id = 1;
-- Session 2: BEGIN; UPDATE accounts SET balance = 0 WHERE id = 2;
-- Session 1: UPDATE accounts SET balance = 0 WHERE id = 2; -- WAITS
-- Session 2: UPDATE accounts SET balance = 0 WHERE id = 1; -- DEADLOCK!
-- PostgreSQL detects and kills one session with error

-- Prevention: always acquire locks in consistent order
```

Thursday — UPSERT and Concurrent Patterns (60 min)

Topics:

- INSERT ... ON CONFLICT DO NOTHING
- INSERT ... ON CONFLICT DO UPDATE SET (upsert)
- Conditional upsert with WHERE clause
- Optimistic concurrency with version columns
- SELECT ... FOR UPDATE NOWAIT pattern

```sql
-- Basic upsert
INSERT INTO products (id, name, price)
VALUES (1, 'Widget', 9.99)
ON CONFLICT (id) DO UPDATE
    SET name  = EXCLUDED.name,
        price = EXCLUDED.price;

-- Only update if new price is lower (conditional upsert)
INSERT INTO products (id, name, price)
VALUES (1, 'Widget', 8.99)
ON CONFLICT (id) DO UPDATE
    SET price = EXCLUDED.price
    WHERE EXCLUDED.price < products.price;

-- ON CONFLICT DO NOTHING (ignore duplicates)
INSERT INTO user_events (user_id, event_name, occurred_at)
```

```
VALUES (100, 'login', NOW())
ON CONFLICT (user_id, event_name, occurred_at) DO NOTHING;

-- Optimistic locking with version column
-- No lock held during "read-modify-write" cycle
SELECT id, balance, version FROM accounts WHERE id = 1;
-- In application: modify balance
UPDATE accounts
SET balance = new_balance, version = version + 1
WHERE id = 1 AND version = 5;  -- only updates if version hasn't changed
-- If 0 rows affected: someone else updated it — retry!

-- NOWAIT: fail immediately if lock not available
BEGIN;
SELECT * FROM orders WHERE id = 1 FOR UPDATE NOWAIT;
-- If another session holds the lock: ERROR: could not obtain lock
-- Application can handle this gracefully instead of waiting indefinitely
COMMIT;
```

Friday — Internals Review + Concurrency Mini-Project (45 min)

Mini-Project: Implement a concurrent task queue.

```
-- Build a production-quality job queue:
-- 1. workers table, jobs table with SKIP LOCKED
-- 2. Function to claim next available job
-- 3. Function to complete or fail a job
-- 4. Query to show queue depth and worker stats
-- 5. Test with 2 simulated workers

-- Bonus: add retry logic (up to 3 attempts)
-- Bonus: add priority queue (priority column, ORDER BY priority DESC)
```

Practice Tasks

1. Demonstrate a non-repeatable read at READ COMMITTED isolation level.
2. Write a deadlock scenario and explain how PostgreSQL resolves it.
3. Inspect dead tuples before and after VACUUM using `pg_stat_user_tables`.
4. Implement an optimistic locking update with retry logic in a PL/pgSQL function.
5. Build a job queue using SKIP LOCKED and process 5 jobs in parallel.
6. Use `pg_advisory_lock` to prevent concurrent execution of a function.
7. Check transaction ID age on your database and understand the wraparound risk.
8. Write a concurrent-safe inventory decrement using `FOR UPDATE NOWAIT`.

Self-Assessment Checklist

- [ ] I can explain MVCC in my own words (no notes)
- [ ] I understand the difference between all four isolation levels
- [ ] I can demonstrate a deadlock and explain prevention strategies

- [ ] I understand what VACUUM does and why autovacuum exists
- [ ] I can write a correct UPSERT with ON CONFLICT
- [ ] I implemented a job queue with SKIP LOCKED
- [ ] I understand transaction ID wraparound risk

Mock Interview Questions

1. Explain MVCC. Why does PostgreSQL use it instead of locking?
2. What is a phantom read? At which isolation level does it occur?
3. You have a hot-standby replica that's falling behind. What could cause this?
4. What happens to dead tuples in PostgreSQL? Why does this matter?
5. Explain transaction ID wraparound. How do you prevent it?
6. When would you use SERIALIZABLE isolation? What are the performance costs?
7. How does SKIP LOCKED work? Build a job queue using it.
8. What is the difference between optimistic and pessimistic locking?

Resources

- This repo: `09_Transactions_Concurrency/`, `10_PostgreSQL_Internals/`
- PostgreSQL concurrency: https://www.postgresql.org/docs/16/mvcc.html
- "PostgreSQL 14 Internals" by Egor Rogov (free PDF)